指针传参能改实参，为什么本质还是"传值"

嵌入式 Linux 手记　2026-10-12　公众号文章 · 原链接 mp.weixin.qq.com

先看两个函数。它们都拿着指针，一个"改成功"了，一个"什么都没改"：

```c
#include <stdio.h>

void set_value(int *p) { *p = 100; }  // 改得动指向的对象
void move_ptr(int *p)  { p++; }       // 改不动指针本身

int main(void)
{
    int a = 1;
    int *q = &a;
    set_value(q);
    printf("a = %d\n", a);            // 100：改成功了
    move_ptr(q);
    printf("q = %p\n", (void*)q);     // 仍等于 &a：纹丝不动
    return 0;
}
```

教科书把前者叫"传址"：传地址进去，函数就能改外面的变量。可 `p++` 同样是在"改指针"，为什么调用者的 `q` 纹丝不动？这两个现象摆在一起，"传址"这个词就露馅了。这篇把谜底揭开：C 语言从来只有一种传参方式。

一、一成一败，差在动的是哪一层

`set_value` 里，`*p = 100` 动的是"p 所指向的对象"。`p` 里存的地址是 `&a` 的副本，凭这个地址写内存，写到的正是 `main` 里的 `a`。副本和原件指向同一处，改对象当然双方可见。

`move_ptr` 里，`p++` 动的是"p 自己"。`p` 是函数的形参，是一个带存储的新变量；自增改的是这个新变量的值，改完的 `p` 指向 `a` 之后的内存，而 `main` 里的 `q` 从头到尾没被碰过。

同样一个指针形参，改"所指向的对象"能穿透回去，改"指针本身"只动了副本。区分这两个层级，靠的正是前几篇反复用的四要素：形参 `p` 自己也是一个对象，类型 `int *`，有 8 字节存储，值是 `&a` 的一份拷贝。



图：两个函数的栈帧对照：`*p = 100` 写穿到调用者的对象；`p++` 只挪动被调栈帧里的副本

二、C 只有传值，"传址"是误称

K&R 第二版 §1.8 写得直接：在 C 里，所有函数参数都是"按值传递"（call by value）。标准给传参的定义等价于一次赋值：进入函数时，每个形参被赋予对应实参的值。传 `&a` 时，赋给形参的"值"恰好是一个地址，如此而已。

用四问拆形参 `p`：类型 `int *`；所指向类型 `int`；值是 `&a` 的副本；自身有 8 字节存储。既然 `p` 是有存储的独立对象，它就可以被修改（`p++`），但修改只发生在它自己的存储上。所谓"传址"，只是传的值恰好是地址；副本与原件存着同一个地址，才有了"能改实参"的效果。

gdb 一分钟能验证这一点：在 `move_ptr` 里 `print &p`，回到 `main` 里 `print &q`，两个地址不同。机器层面更直观：arm64 上实参经 `x0` 寄存器传入，被调方在函数开头（prologue）把寄存器值存进自己的栈帧槽位，此后 `p++` 改的是那个槽位；调用者的 `q` 躺在 `main` 的栈帧里，谁也够不着谁。

顺带划清边界：真正"形参是实参别名"的传引用是 C++ 的特性。C 里没有引用，任何"能改实参"的效果，都来自手里握着对方的地址。

为什么 C 要选传值？这个设计与赋值语义一脉相承：实参就像形参的初始值，函数调用没有隐藏的别名，每个副作用都写在明面上——想改外部，必须显式取地址，调用点一眼可见。代价是写法稍繁，收益是副作用零意外。还有一层容易被忽视的推论：副本的成本固定，无论传 `int` 还

是传指针，复制的都只有那一份。有人担心"传指针比传值慢"，方向恰恰反了：传大结构体才要复制一大块，传指针只复制 8 字节。把地址当值来传，正是 C 兼顾语义与开销的设计。

三、想改指针本身？再加一级

最常见的翻车现场是链表头插。先看错误直觉：

```
void push_wrong(struct Node *head, int v)
{
    struct Node *n = malloc(sizeof *n);
    if (n == NULL) return;
    n->val = v;
    n->next = head;
    head = n;           // 只挪了副本，函数返回后调用者的 head 仍是 NULL
}
```

函数内部一切正常，`head` 确实指向了新节点；可它是个副本，返回即销毁。调用者的 `head` 还是 `NULL`，新节点成了没有任何指针指向它的孤儿，内存泄漏就此发生。编译器对这段代码一声不吭。

修正版把形参升一级：

```
void push(struct Node **head, int v)
{
    struct Node *n = malloc(sizeof *n);
    if (n == NULL) return;
    n->val = v;
    n->next = *head;
    *head = n;          // 凭"地址的副本"写调用者的指针本身
}

/* 调用方 */
push(&head, 1);         // 传入的是 head 的地址
```

`head` 的副本里存的是 `&head`，`*head = n` 凭这个地址写到了调用者的指针变量上。这一幕和篇 3 的两层图完全同构：副本落在哪一层，写入就穿到哪一层。要让"写"穿透到指针变量本身，手里就得握着指针变量的地址，也就是指针的指针。



*图：链表头插动画：副本存着 &head，*head 写穿替换调用者的指针；push_wrong 的副本返回即丢弃*

另一个选择是返回新值：`head = push(head, v)`，让函数返回新链表头，调用者自己接住。两种写法等价，工程里都常见，选一种全库统一即可。这个模式在标准库也随处可见：`scanf("%d", &n)` 必须收 `&n` 才能写入 `n`；`frexp(x, &exp)` 用输出参数带回指数。Linux 内核的返回约定也建立在这条语义上：函数返回错误码，结果经输出参数带回，如 `kstrtoint(s, base, &res)`，调用者传 `res` 的地址，函数凭副本写回。读内核源码时看到成片的输出参数，根子都在"传值"二字。

反过来看也成立：`swap(a, b)` 直接传两个 `int` 为什么换不动？因为函数拿到的是副本，`x`、`y` 在里面怎么交换都到不了调用者手里。初学时都背过"swap 要传地址"，现在可以说清原理了：语言没有对指针开后门，只是地址本身成了被复制的那份值。

写在最后

把这篇压成五条：

1. C 只有一种传参方式：传值。所谓传址，只是传的值恰好是地址。
2. 形参是带存储的副本：改 *p 能影响调用者（写的是双方共同指向的对象），改 p 无效（写的是副本自己）。
3. 函数要替换调用者的指针，两条正路：二级指针形参 `push(&head, v)`，或返回新值 `head = push(head, v)`。别在函数里改一级指针形参。
4. 把 `malloc` 结果赋给一级指针形参然后"等它生效"，是泄漏的经典来源；编译器不会报错，只能靠这条意识拦住。

5. 拿不准就 gdb：函数内外分别 `print &p` 与 `&q`，地址不同即副本；arm64 上还能在 prologue 处看到寄存器值入栈。

下一篇回到数组：`a = b` 为什么非法？"能被赋值的东西"在语言里有个正式名字，叫可修改左值。下一篇讲左值判定，把数组与结构体在赋值上的差别一次讲透。

合集持续更新，往期与后续篇目都在目录里：C 语言底层实战 · 合集目录

后台回复「交流」，我很乐意分享我的学习经验和工作经验，直接来聊。